

Python Tuple(Module-7)

Tuples are used to store multiple items in a single variable. Tuple is one of 4 built-in data types in Python used to store collections of data, the other 3 are List, Set, and Dictionary, all with different qualities and usage. A tuple is a collection which is ordered and **unchangeable**. Tuples are written with round brackets.

```
thistuple = ("apple", "banana", "cherry")
print(thistuple)
```

Output:

```
('apple', 'banana', 'cherry')
```

Tuple Items

Tuple items are ordered, unchangeable, and allow duplicate values. Tuple items are indexed, the first item has index [0], the second item has index [1] etc.

Ordered

When we say that tuples are ordered, it means that the items have a defined order, and that order will not change.

Unchangeable

Tuples are unchangeable, meaning that we cannot change, add or remove items after the tuple has been created.

Allow Duplicates

Since tuples are indexed, they can have items with the same value.

Tuple Length

To determine how many items a tuple has, use the len() function:

```
thistuple = ("apple", "banana", "cherry")
print(len(thistuple))
```

Output:

```
3
```

Creating empty tuple

To create an empty tuple we can use empty parenthesis.

```
t= ()
```

Create Tuple with One Item

To create a tuple with only one item, you have to add a comma after the item, otherwise Python will not recognize it as a tuple.

One item tuple, remember the comma:

```
thistuple = ("apple",)
print(type(thistuple))
```

```
#NOT a tuple
thistuple = ("apple")
print(type(thistuple))
```

Output:

```
<class 'tuple'>
<class 'str'>
```

The tuple() Constructor

It is also possible to use the tuple() constructor to make a tuple.

```
thistuple = tuple(("apple", "banana", "cherry")) # note the double round-brackets
print(thistuple)
```

Output:

```
('apple', 'banana', 'cherry')
```

Creating tuple from existing sequence

```
l= [1, 2, 3, 4, 5]
t= tuple (l)
print (t)
```

Output:

```
(1, 2, 3, 4, 5)
```

Nested tuple

```
t= (1, 2, (3, 4, 5), 6, 7)
print (t[2])
```

Output:

```
(3, 4, 5)
```

Access Tuple Items

You can access tuple items by referring to the index number, inside square brackets:

```
thistuple = ("apple", "banana", "cherry")
print(thistuple[1])
```

Output:

```
banana
```

Negative Indexing

Negative indexing means start from the end. -1 refers to the last item, -2 refers to the second last item etc.

```
thistuple = ("apple", "banana", "cherry")
print(thistuple[-1])
```

Output:

cherry

Change Tuple Values

Once a tuple is created, you cannot change its values. Tuples are **unchangeable**, or **immutable** as it also is called. You can convert the tuple into a list, change the list, and convert the list back into a tuple.

```
x = ("apple", "banana", "cherry")
y = list(x)
y[1] = "kiwi"
x = tuple(y)
print(x)
```

Output:

("apple", "kiwi", "cherry")

Similarly you can add or delete items from the tuple by converting it into a list.

Deleting Tuple

```
thistuple = ("apple", "banana", "cherry")
del thistuple
```

Unpacking a Tuple

```
fruits = ("apple", "banana", "cherry")
green, yellow, red = fruits
print(green)
print(yellow)
print(red)
```

Output:

apple
banana
cherry

Using Asterisk*

If the number of variables is less than the number of values, you can add an * to the variable name and the values will be assigned to the variable as a list:

```
fruits = ("apple", "banana", "cherry", "strawberry", "raspberry")
(green, yellow, *red) = fruits
print(green)
print(yellow)
print(red)
```

Output:

apple
banana

```
['cherry', 'strawberry', 'raspberry']
```

```
fruits = ("apple", "mango", "papaya", "pineapple", "cherry")  
(green, *tropic, red) = fruits  
print(green)  
print(tropic)  
print(red)
```

Output:

```
apple  
['mango', 'papaya', 'pineapple']  
cherry
```

Join Two Tuples

To join two or more tuples you can use the + operator:

```
tuple1 = ("a", "b", "c")  
tuple2 = (1, 2, 3)  
tuple3 = tuple1 + tuple2  
print(tuple3)
```

Output:

```
('a', 'b', 'c', 1, 2, 3)
```

Multiply Tuples

If you want to multiply the content of a tuple a given number of times, you can use the * operator:

```
fruits = ("apple", "banana", "cherry")  
mytuple = fruits * 2  
print(mytuple)
```

Output:

```
('apple', 'banana', 'cherry', 'apple', 'banana', 'cherry')
```

Length of Tuple

We can find out the length of the tuple using len() function.

```
fruits = ("apple", "banana", "cherry")  
print(len(fruits))
```

Output:

```
3
```

Finding maximum and minimum value

We can use max() and min() functions respectively to find out the maximum and minimum value from a tuple.

```
t=(10, 5, 7, 19, 17)
```

```
print(max(t),min(t))
```

Output:

19 5

Tuple Methods

count()

The count() method returns the number of times a specified value appears in the tuple.

Syntax

```
tuple.count(value)
```

```
thistuple = (1, 3, 7, 8, 7, 5, 4, 6, 8, 5)
x = thistuple.count(5)
print(x)
```

Output:

2

index()

The index() method finds the first occurrence of the specified value. The index() method raises an exception if the value is not found.

Syntax

```
tuple.index(value)
```

```
thistuple = (1, 3, 7, 8, 7, 5, 4, 6, 8, 5)
x = thistuple.index(8)
print(x)
```

Output:

3